

# RELATÓRIO TÉCNICO DE CONTINUIDADE OPERACIONAL E MONITORIZAÇÃO



**Formando:**Odair Santos Mota

**Formador:** Valério Santos

**Modulo:**Linux e Segurança na Cloud

## Índice

|                                                                             |   |
|-----------------------------------------------------------------------------|---|
| Plano de Monitorização, Backup e Continuidade Operacional – Tópico 05 ..... | 1 |
| 1. Identificação do Grupo e Autoria .....                                   | 1 |
| 2. Serviço Analisado e Caracterização Técnica .....                         | 1 |
| 3. Plano de Monitorização Proposto .....                                    | 2 |
| 4. Plano de Auditoria de Logs .....                                         | 3 |
| 5. Plano de Backup (Política de Salvaguarda) .....                          | 4 |
| 6. Plano de Recuperação Simples (Disaster Recovery) .....                   | 6 |
| 7. Matriz de Riscos e Medidas de Mitigação .....                            | 7 |
| 8. Conclusão Final .....                                                    | 8 |

# Plano de Monitorização, Backup e Continuidade Operacional – Tópico 05

## 1. Identificação do Grupo e Autoria

- **Identificação do Grupo:** Trabalho Adaptado para Regime Individual
- **Autor / Formando:** Odair Santos Mota
- **Perfil Profissional / Papéis Assumidos:** Engenheiro de Sistemas, Administrador Cloud LAMP e Especialista em Continuidade de Negócio.

## 2. Serviço Analisado e Caracterização Técnica

- **Cenário Selecionado:** Cenário C – Sistema de Gestão de Conteúdos (CMS) **WordPress** publicado sobre Servidor Web **Apache** e Base de Dados **MariaDB** (Stack LAMP).
- **Componentes Críticos:** Daemon do Apache (apache2), motor de base de dados (mariadb), interpretador PHP e o sistema de ficheiros persistente.
- **Ficheiros e Configurações Importantes:** Ficheiro de conexão e segredos do CMS (wp-config.php), diretivas de hosts virtuais do Apache (/etc/apache2/sites-available/000-default.conf) e a diretoria raiz de dados (/var/www/html/wordpress/).
- **Riscos Principais de Indisponibilidade:** Esgotamento de memória RAM devido a picos de tráfego, paragem do serviço de base de dados por corrupção de tabelas, saturação de armazenamento em disco por crescimento descontrolado de ficheiros temporários, e ataques de negação de serviço (DoS/DDoS).

### 3. Plano de Monitorização Proposto

Estratégia proativa para acompanhar a saúde e os limites operacionais do mini-servidor Linux através de comandos nativos e métricas de desempenho:

| Elemento a Monitorizar | Como Verificar (Comando Linux)                          | Frequência Sugerida                    | Evidência Operacional                                                            |
|------------------------|---------------------------------------------------------|----------------------------------------|----------------------------------------------------------------------------------|
| Estado do Serviço Web  | <code>sudo systemctl status apache2</code>              | A cada 5 minutos (Automatizado)        | Estado active (running) no terminal ou código HTTP 200 via script.               |
| Espaço em Disco        | <code>df -h /</code>                                    | Diária (Alerta se > 85%)               | Percentagem de uso da partição raiz visível na coluna Use%.                      |
| Memória RAM            | <code>free -h</code>                                    | A cada 10 minutos                      | Monitorização de RAM disponível e buffer/cache para evitar o <i>OOM Killer</i> . |
| Logs de Erro           | <code>sudo tail -n 20 /var/log/apache2/error.log</code> | De hora em hora (Análise de anomalias) | Presença de códigos de erro do motor                                             |

| <b>Elemento a Monitorizar</b> | <b>Como Verificar (Comando Linux)</b> | <b>Frequência Sugerida</b>           | <b>Evidência Operacional</b>                                      |
|-------------------------------|---------------------------------------|--------------------------------------|-------------------------------------------------------------------|
|                               |                                       |                                      | PHP ou falhas de timeout.                                         |
| <b>Autenticação (SSH)</b>     | sudo tail -n 20 /var/log/auth.log     | Tempo Real (Via ferramentas de SIEM) | Registos de Accepted publickey ou Failed password para auditoria. |

#### 4. Plano de Auditoria de Logs

Mapeamento dos ficheiros de registo críticos para análise forense, diagnóstico de segurança e deteção de falhas na aplicação:

| <b>Log do Sistema</b> | <b>Finalidade Primária</b>                                               | <b>Evento Relevante a Identificar</b>                        |
|-----------------------|--------------------------------------------------------------------------|--------------------------------------------------------------|
| journalctl            | Monitorizar o estado geral do kernel e inicialização de daemons.         | Falhas de hardware virtual ou paragens abruptas de serviços. |
| auth.log              | Auditar acessos administrativos e tentativas de elevação de privilégios. | Utilização bem-sucedida ou rejeitada do comando sudo.        |

| <b>Log do Sistema</b> | <b>Finalidade Primária</b>                                             | <b>Evento Relevante a Identificar</b>                                          |
|-----------------------|------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| access.log            | Registar todas as requisições HTTP dirigidas ao ecossistema WordPress. | Injeções de tráfego malicioso (ex: tentativas de brute force em wp-login.php). |
| error.log             | Capturar falhas críticas de infraestrutura web e quebras de ligação.   | Erros internos do servidor (HTTP 500) ou falha de comunicação com MariaDB.     |

## 5. Plano de Backup (Política de Salvaguarda)

A política baseia-se na redundância de dados para evitar pontos únicos de falha, dividindo o ecossistema LAMP em camadas isoladas:

| <b>Item Crítico</b>      | <b>Backup Necessário?</b> | <b>Frequência</b>                              | <b>Local e Retenção Mínima</b>                               | <b>Justificação Técnica</b>                                                       |
|--------------------------|---------------------------|------------------------------------------------|--------------------------------------------------------------|-----------------------------------------------------------------------------------|
| <b>Ficheiros do Site</b> | <b>SIM</b>                | Diário (Incremental)<br><br>Semanal (Completo) | Servidor NAS Externo / Cloud.<br>Retenção mínima de 30 dias. | Garante a recuperação total de imagens, plugins e ficheiros do core do WordPress. |
| <b>Configuração Web</b>  | <b>SIM</b>                | Sempre que alterado                            | Repositório Git Privado e                                    | Permite reconfigurar                                                              |

| Item Crítico  | Backup<br>Necessário? | Frequência                      | Local<br>Retenção<br>Mínima                                                 | Justificação<br>Técnica                                                                             |
|---------------|-----------------------|---------------------------------|-----------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
|               |                       |                                 | Backup do SO.<br>Retenção<br>estável<br>(Histórico).                        | os ficheiros<br>.conf do<br>Apache em<br>segundos<br>numa nova<br>VM.                               |
| Base de Dados | SIM                   | A cada 6<br>horas (Dump<br>SQL) | Disco Separado<br>+<br>Armazenamento<br>Externo.<br>Retenção de 14<br>dias. | Camada<br>mais volátil.<br>Armazena<br>utilizadores,<br>posts,<br>páginas e<br>metadados<br>vitais. |
| Documentação  | SIM                   | Sempre que<br>atualizado        | Repositório<br>Centralizado de<br>Gestão de<br>Conhecimento.                | Garante que<br>a equipa<br>técnica sabe<br>operar a<br>infraestrutura<br>em caso de<br>desastre.    |

**Validação do Backup:** Executada semanalmente pelo Administrador de Sistemas através de testes de restauro automatizados em ambiente de *Staging* (Homologação).

## 6. Plano de Recuperação Simples (Disaster Recovery)

### Cenário A: A página web desapareceu (Erro 404 ou Diretório Apagado)

- **Procedimento:** Verificar a integridade do diretório `/var/www/html/wordpress/`. Se os ficheiros foram eliminados acidentalmente ou corrompidos, restaurar o último backup dos ficheiros do site utilizando o comando: `sudo tar -xvpzf /backups/wordpress_completo.tar.gz -C /var/www/html/`.

### Cenário B: O serviço web Apache parou abruptamente

- **Procedimento:** Verificar os logs de erro imediatos com `sudo journalctl -u apache2 -n 50`. Tentar reiniciar o serviço de forma segura através do comando `sudo systemctl restart apache2`. Se a falha persistir devido a erro de configuração, reverter o ficheiro `/etc/apache2/sites-available/000-default.conf` para a versão de produção guardada no Git.

### Cenário C: O disco rígido do servidor ficou cheio (100% de Ocupação)

- **Procedimento:** Localizar ficheiros volumosos indesejados através do comando `sudo du -sh /var/log/*`. Executar a limpeza segura de pacotes temporários acumulados (`sudo apt clean`), rotacionar logs antigos compactados (gzip) e mover dumps de bases de dados antigos para o armazenamento de longo prazo.

### Validação da Recuperação e Evidências

- **Validação:** A recuperação é validada executando `systemctl is-active` para os daemons e realizando um teste de conectividade externa via browser ou comando `curl -I http://localhost` para verificar o retorno do cabeçalho HTTP/1.1 200 OK.
- **Evidências a Guardar:** Outputs de logs pós-incidente, o timestamp do ficheiro de backup utilizado no restauro e o relatório de incidentes preenchido.



## 7. Matriz de Riscos e Medidas de Mitigação

Mapeamento dos perigos de infraestrutura e respostas operacionais planeadas:

| Risco Identificado               | Impacto Operacional | Medida de Mitigação / Resposta Técnica                                                                                |
|----------------------------------|---------------------|-----------------------------------------------------------------------------------------------------------------------|
| Corrupção da Base de Dados       | ALTO                | Agendamento via cron de rotinas automáticas de reparação com mysqlcheck e dumps periódicos em servidores isolados.    |
| Esgotamento Crítico de Disco     | ALTO                | Configuração rigorosa do utilitário logrotate para compactar e expurgar registos com mais de 7 dias.                  |
| Ataque de Força Bruta no SSH/Web | MÉDIO               | Instalação e parametrização do Fail2Ban para bloquear automaticamente IPs com mais de 3 tentativas falhadas de login. |
| Falha de Hardware da VM          | CRÍTICO             | Exportação mensal de Snapshots completos da máquina virtual para um hipervisor secundário de backup.                  |

## **8. Conclusão Final**

A estabilidade de um ecossistema assente na Stack LAMP depende inteiramente da capacidade da equipa técnica em antecipar cenários de desastre. A aplicação integrada de uma rotina rigorosa de monitorização (trazendo visibilidade ao consumo de RAM e disco) combinada com uma política de cópias de segurança redundantes transforma um servidor vulnerável numa infraestrutura altamente resiliente. As diretivas descritas neste plano garantem que, mesmo perante falhas severas ou corrupção de dados, o serviço web WordPress possa ser restabelecido com o mínimo impacto para o utilizador e zero perda de dados críticos.